In The Name of God

Neural Networks and Deep Learning (2026)

Assignment (3)

Professor: Dr. Mansoori

Student: Paniz Azarnia 40435097

---

1 Conditional GAN on Fashion-MNIST

- ### Short explanation of cGAN architecture

  In this project, I implemented a Conditional Generative Adversarial Network (cGAN) consisting of two subnetworks: a Generator and a Discriminator. Both networks receive class information in addition to their normal inputs.

  #### Generator:

  The generator takes a random noise vector and a one-hot-like label embedding as input.
  These are concatenated and passed through several fully connected layers with BatchNorm and LeakyReLU activations.
  The output is a 28×28 grayscale Fashion-MNIST image conditioned on the requested class.

  #### Discriminator:

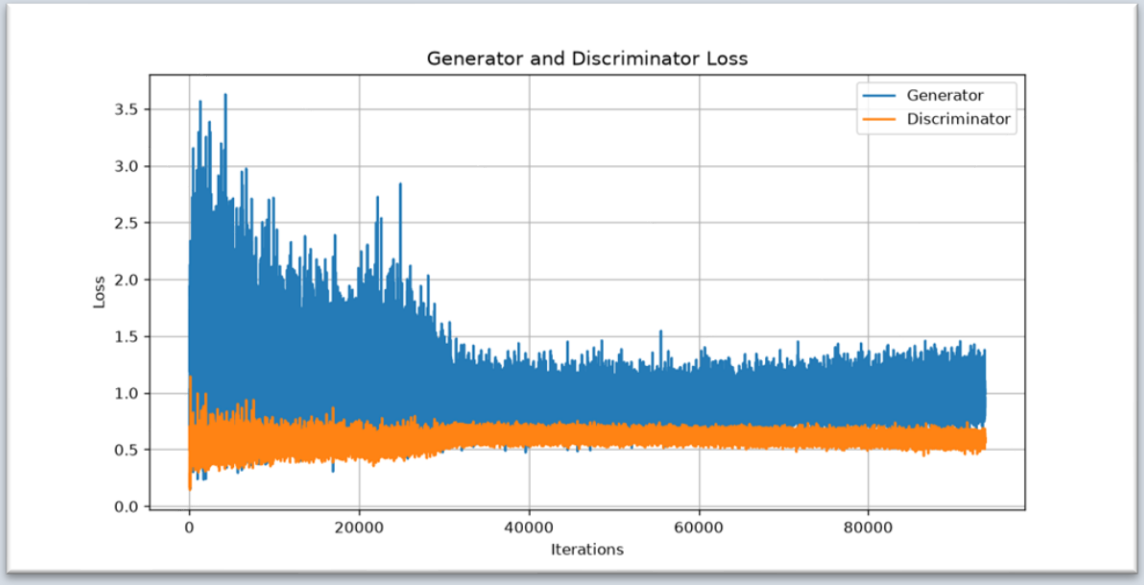
  The discriminator receives a flattened image together with a label embedding.
  These are concatenated and passed through multiple fully connected layers to predict whether the input image is real or fake.
  This architecture allows the GAN to generate images that belong to specific Fashion-MNIST classes, rather than random, unconditional images.

- ### Generated image grid showing samples from different Fashion-MNIST classes

  #### For 100 epochs:



- Plot of generator loss and discriminator loss



- How did the generator and discriminator losses behave?

  High Volatility at the Start: In the first 20,000-30,000 iterations, the Generator loss (blue) shows massive spikes. This is the "discovery" phase where the generator is wildly trying different patterns to see what sticks.

  Adversarial Equilibrium: After about 30,000 iterations, the losses stabilize. The Discriminator loss (orange) stays consistently around 0.5 - 0.8, which is a "sweet spot" in GAN training. It means the discriminator is neither too strong nor too weak.

  Generator Convergence: The Generator loss settles into a steady range (between 0.8 and 1.5). The fact that the losses don't collapse to zero is a good sign; it indicates a healthy competition where both networks are still learning from each other.

- What is the main difference between a GAN and a cGAN?

  A standard GAN only takes random noise as input and generates unconditional images.
  A conditional GAN (cGAN) receives extra information, usually a class label, in addition to noise.
  As a result:
  GAN → generates images without any control over the content.
  cGAN → generates images of specific classes, such as sneakers, shirts, or bags.

- How did you provide class information to the generator and discriminator?

  1. For the Generator:

  The goal of the Generator is to transform a random noise vector (z) into an image that corresponds to a specific category (y).

  Embedding Layer: I used an nn.Embedding layer to convert the class index (a number from 0 to 9) into a continuous vector (embedding).

  Concatenation: This label embedding was then concatenated directly with the latent noise vector (z).

  Transformation: The combined vector (Noise + Label info) was passed through the fully connected layers. This ensures that every layer of the Generator is "conditioned" on the specific class it is trying to create.

  2. For the Discriminator:

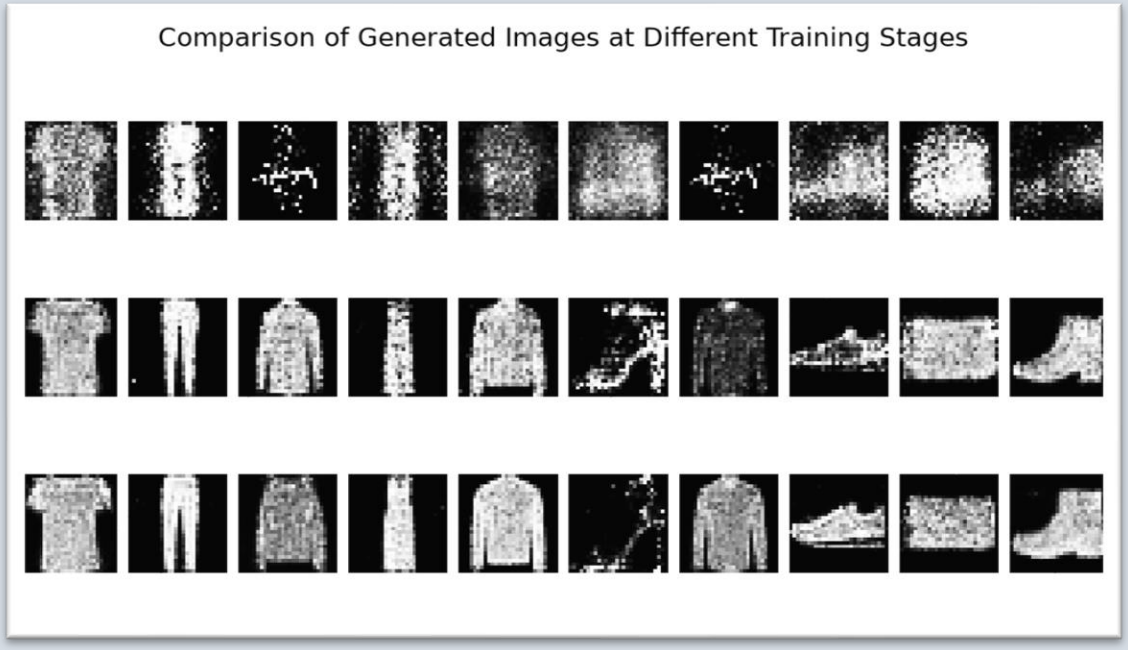
  The Discriminator's job is to determine if an image is real or fake, given a specific class label.

  Embedding Layer: Similar to the generator, the class label was converted into an embedding vector.

Concatenation: The input image (after being flattened) was concatenated with the label embedding.

Decision: By feeding both the image and the label together, the Discriminator learns to check not just if the image "looks real," but specifically if it looks like a real example of the provided class. For example, if the input is a real image of a Bag but the label provided is Sneaker, the Discriminator will correctly classify it as "Fake" (or mismatched).

- How did the generated images change during training?



Early Stage: The images are mostly white noise and random pixels. While some faint silhouettes are visible, the network hasn't yet learned the specific geometry of different clothing items.

Mid Stage: The global structure of the objects emerges. You can clearly distinguish between "long" items (trousers/dresses) and "blocky" items (bags/shirts). However, the images are still quite grainy and lack fine details.

Final Stage: The images become significantly sharper and more defined. The edges are cleaner, and the internal textures (like the sleeves of a shirt or the sole of a sneaker) become visible. This shows that the Generator successfully learned the distribution of the Fashion-MNIST dataset over the 100 epochs.

- Which classes were easier or harder to generate?



Easier Classes: Trousers, Shirts, Pullovers and Ankle Boots are the most successful. These classes have relatively clear and consistent visual structures. For example, trousers usually have a long vertical shape, bags often have a simple rectangular or trapezoidal outline, and ankle boots have a recognizable compact shoe-like shape. Because these classes have more stable silhouettes, the generator was able to capture their main characteristics more successfully.

Harder Classes: On the other hand, Sneakers and Coats were harder to generate. Sneakers contain more fine-grained details, such as soles, openings, and curved boundaries, which are difficult for a simple fully connected GAN to reproduce clearly. Coats were also more challenging because their overall shape can look similar to shirts and pullovers, making it harder for the model to learn class-specific details. As a result, these generated samples sometimes looked less sharp or visually ambiguous.

Overall, the model performed better on classes with simple and consistent shapes, while classes requiring more detailed structure or having overlap with other categories were more difficult to generate.

- How could the results be improved?

  While the current results are good for a fully connected (Linear) architecture, we could reach "near-perfect" quality by:

  Switching to DCGAN: Replacing the nn.Linear layers with nn.Conv2d and nn.ConvTranspose2d (Convolutional layers) would allow the model to understand spatial relationships much better, reducing the "grainy" noise.

  Label Smoothing: Instead of using hard labels (1 for real, 0 for fake), using values like 0.9 for real images can help stabilize the discriminator.

  Increasing Latent Dim: Moving from a latent dimension of 100 to something like 128 or 256 might allow the generator to capture more complex features of the clothing items.

  Learning Rate Decay: Implementing a scheduler to reduce the learning rate in the later stages of training could help the loss converge even more smoothly, leading to sharper final images.

---

2 Grayscale Image Colorization using Convolutional Autoencode

- Short explanation of your Convolutional Autoencoder architecture

  I used a Convolutional Autoencoder (CAE) consisting of two main parts:

  Encoder: takes a 1-channel grayscale image and gradually compresses it into a compact latent representation using a stack of Conv2d layers with stride=2 (downsampling). Each block includes Convolution + Batch Normalization + ReLU, which helps stable training and faster convergence. Also last two layers include dropout for generalization.

  Decoder: reconstructs a 3-channel RGB image from the latent representation using ConvTranspose2d layers with stride=2 (upsampling). Each block uses BatchNorm + ReLU, and the final layer uses Sigmoid so outputs stay in the range [0,1], matching normalized image tensors.

  For input size 128×128, the encoder reduces spatial resolution step-by-step (down to 4×4), while increasing feature channels (from 32 up to 512). The decoder mirrors this process to return to 128×128 and produce 3 output channels (RGB).

- Description of the preprocessing steps, including resizing, normalization, and grayscale conversion

  1.Load image and convert to RGB: Each image is loaded with PIL and converted using:

  ```
  img_rgb = Image.open(img_path).convert("RGB")
  ```

2.Resizing: Both input and target images are resized to a fixed shape 128×128 using:

transforms.Resize((img_size, img_size))

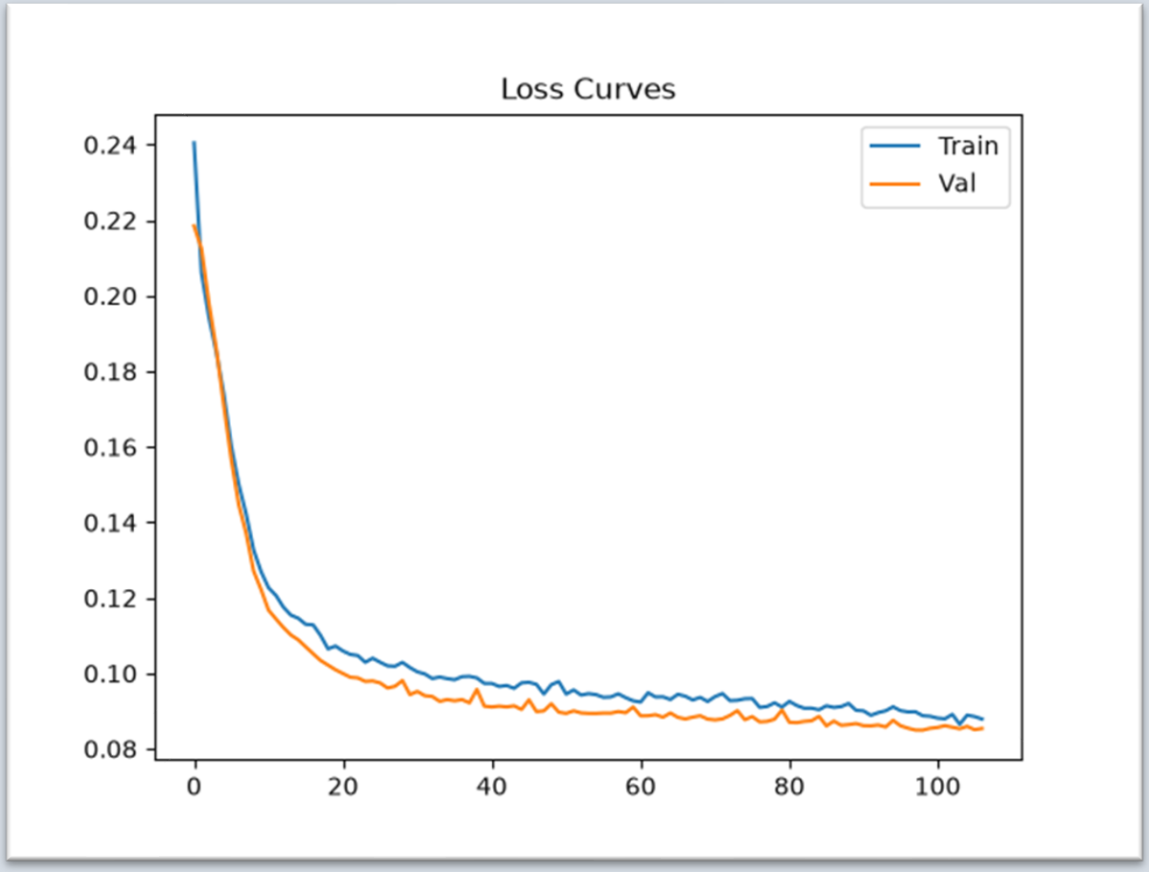
3.Grayscale conversion for the input: The model input is created by converting the RGB image to grayscale:

transforms.Grayscale(num_output_channels=1)

This produces a tensor of shape $(1,H,W)$.

4.Normalization / scaling: transforms.ToTensor() converts images into float tensors in the range [0,1]. This matches the model's Sigmoid output range. So:
Input: grayscale tensor (1,128,128) in [0,1]
Target: RGB tensor (3,128,128) in [0,1]

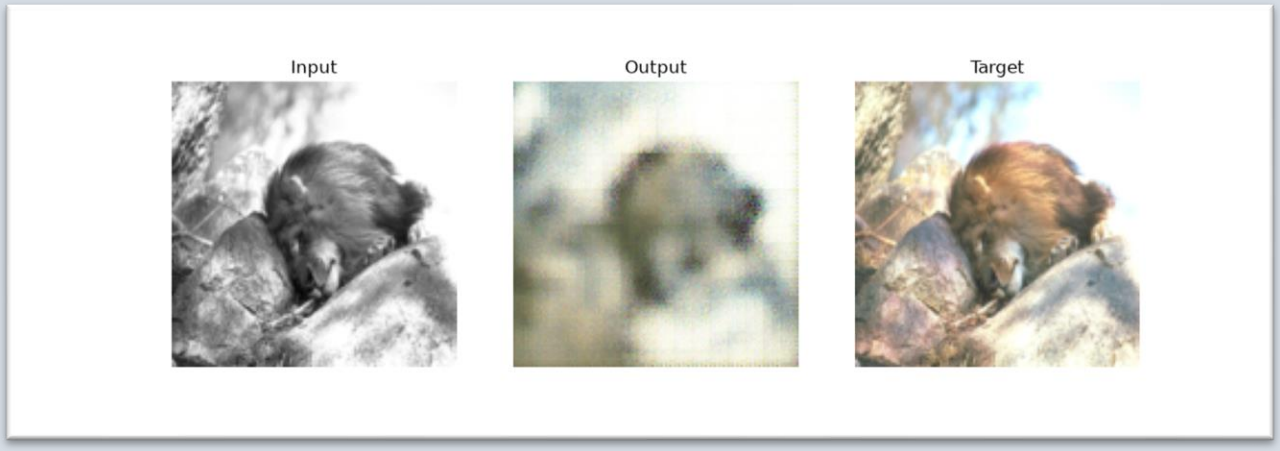
- A plot of training and validation loss.

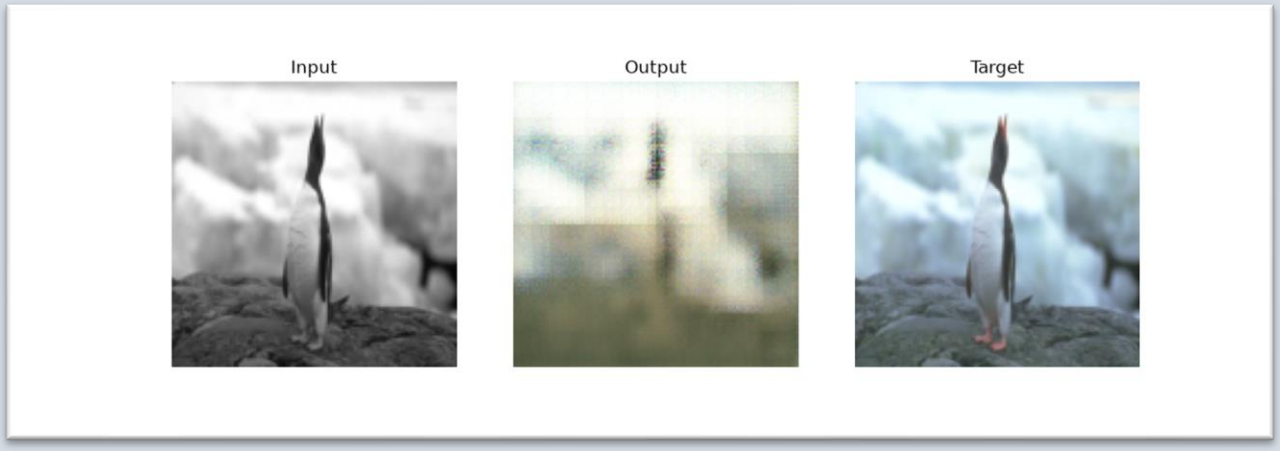


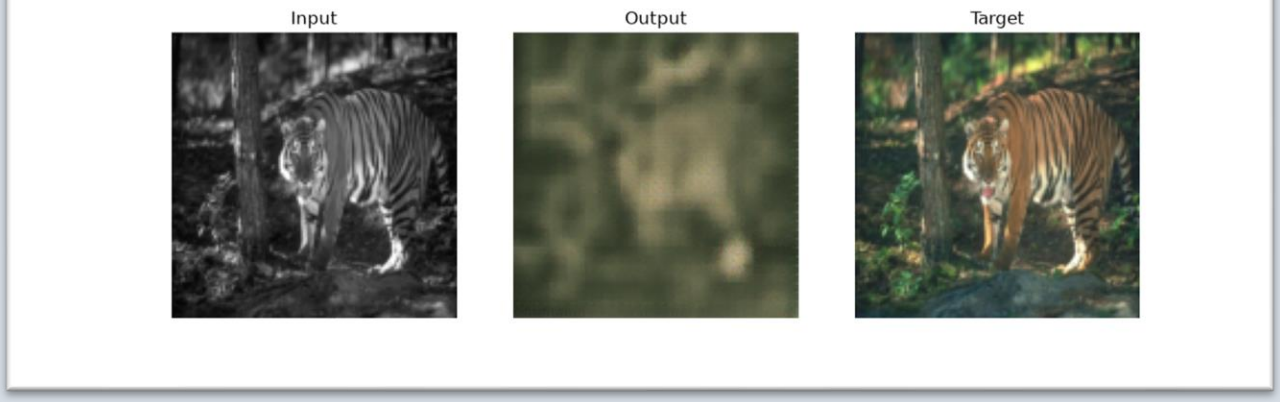
The training and validation loss curves decrease steadily and remain close to each other, indicating stable learning and good generalization. No strong overfitting is observed, although the improvement becomes smaller in later epochs, suggesting convergence

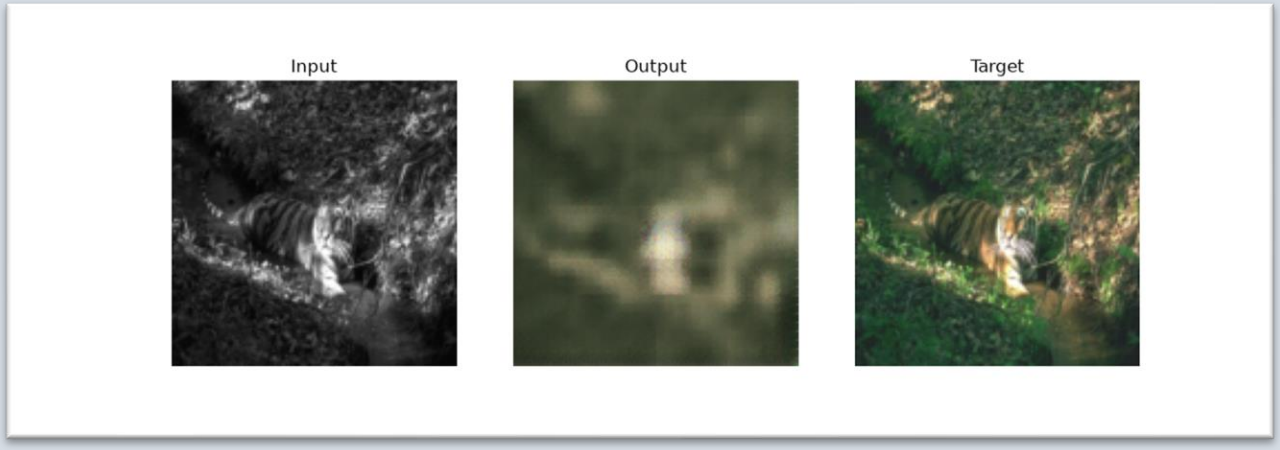
This happened with num_epochs = 200(trained for 107 epochs because of early stopping with patience = 8), batch_size = 16 and lr = 0.0001

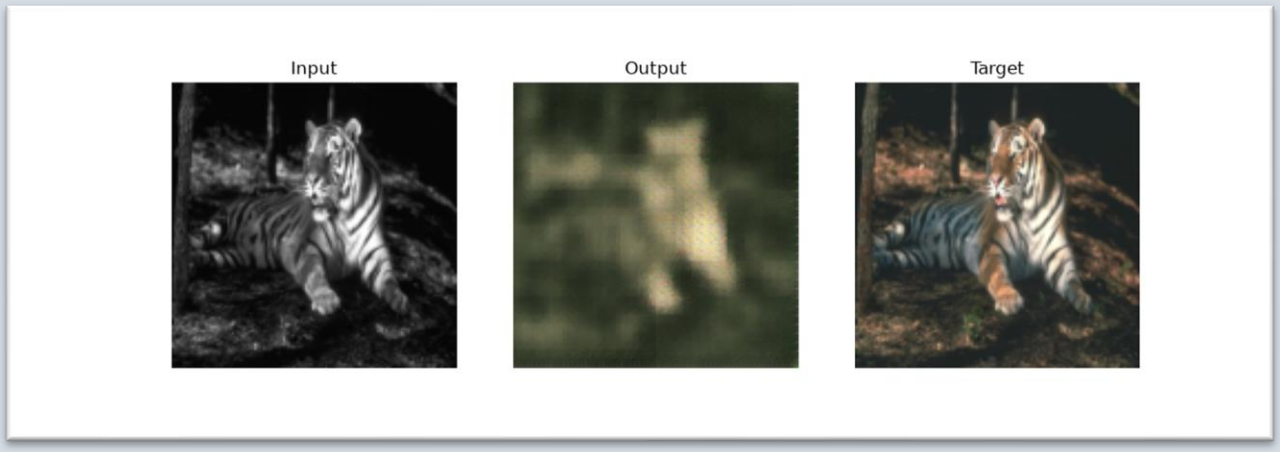
- A visualization of the 10 test results:

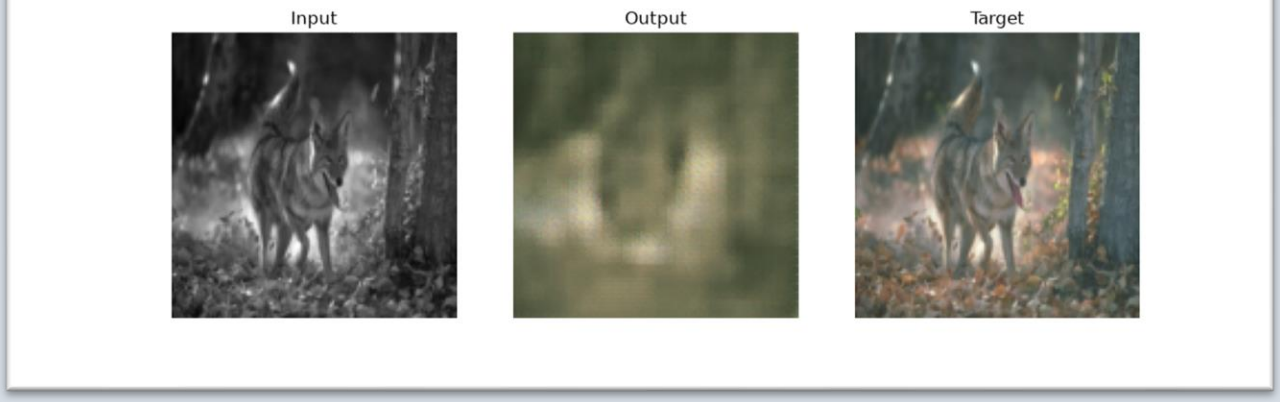


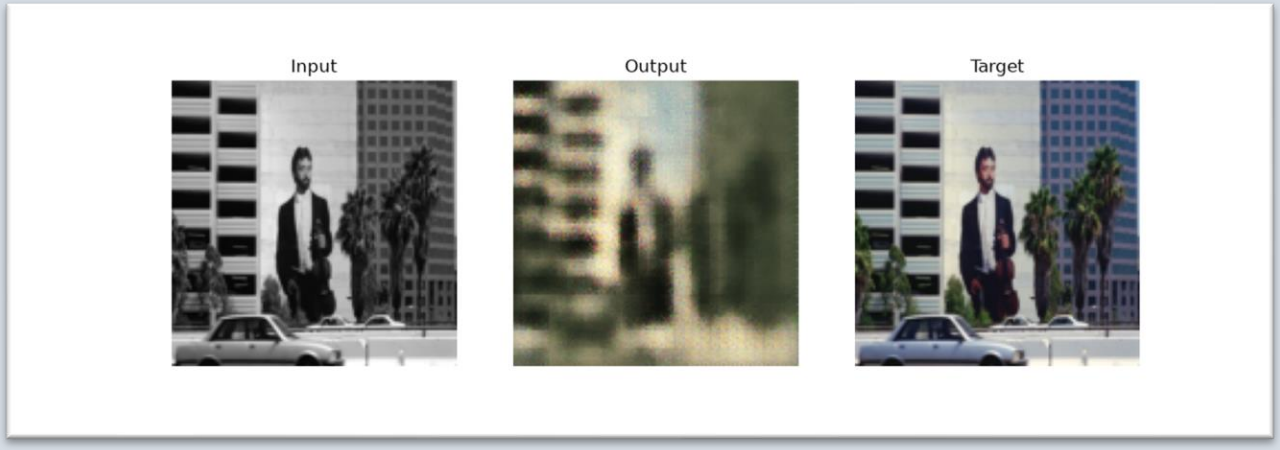


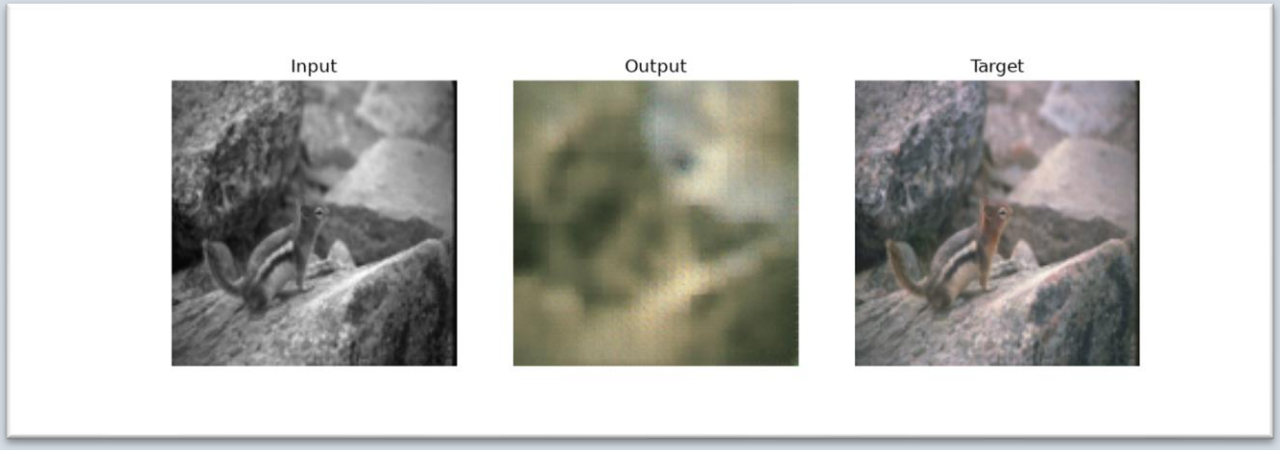

Input
Output
Target

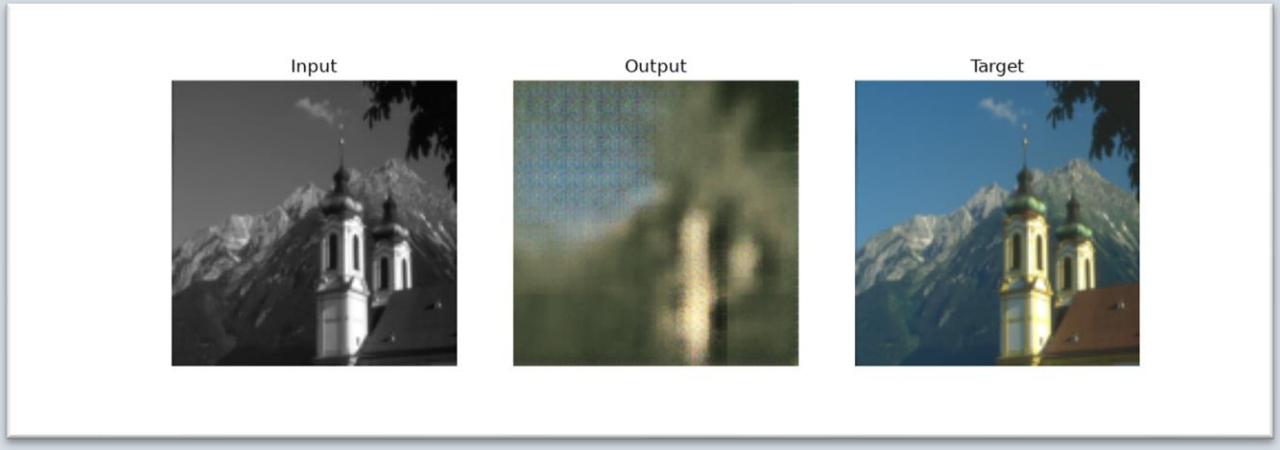

Input
Output
Target

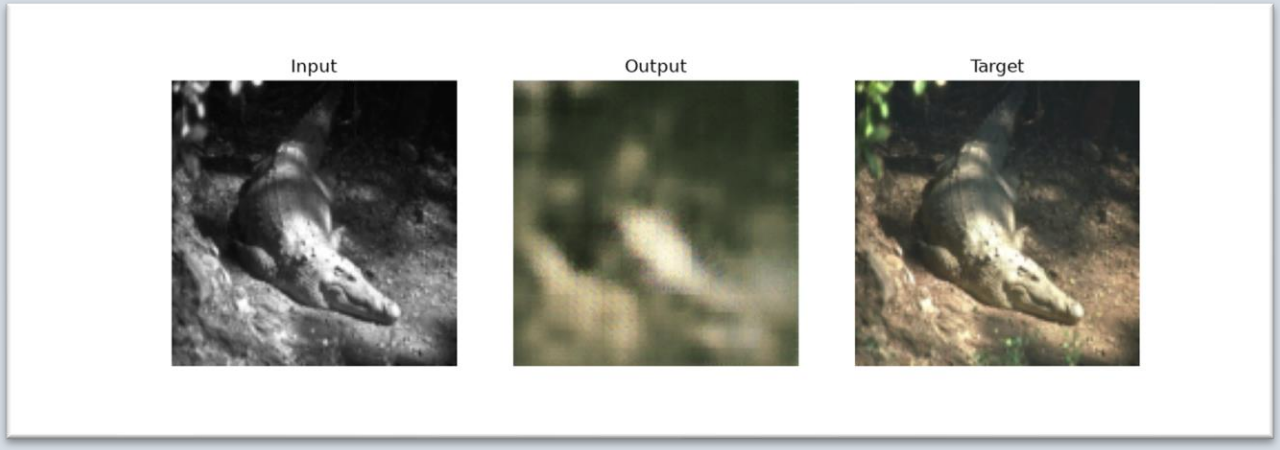

Input
Output
Target


Input
Output
Target


Input
Output
Target


Input
Output
Target





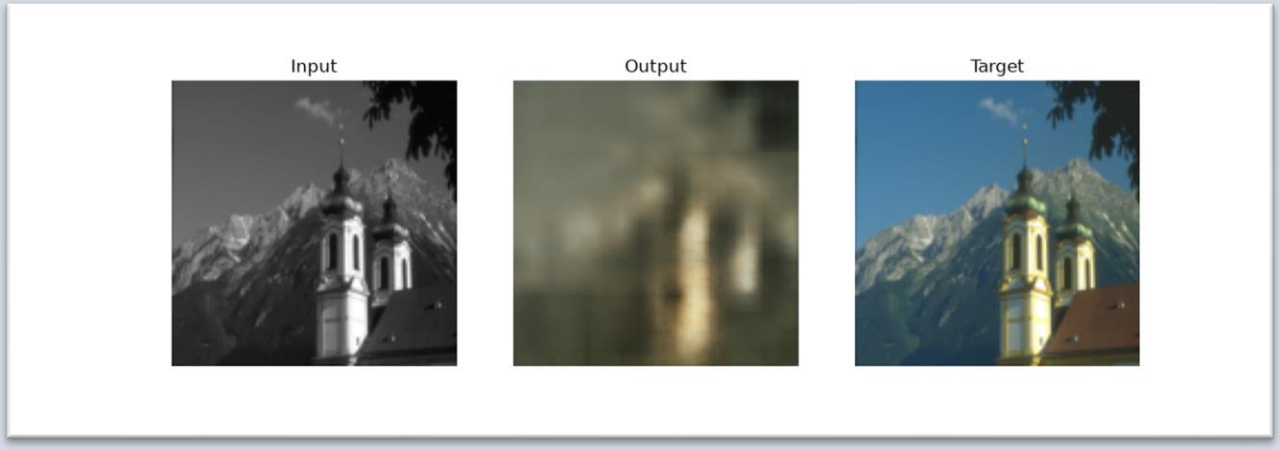
- test-set evaluation metrics:

Metric	*Value*	*Interpretation*
MSE (Mean Squared Error)	0.010969	Measures pixel-wise average squared difference. A value near 0.010 indicates the predicted colors are globally close to the ground truth.
PSNR (Peak Signal-to-Noise Ratio)	19.92 db	A logarithmic metric of reconstruction quality. Values around 20 dB are highly acceptable for standard Convolutional Autoencoders on small colorization datasets.
SSIM (Structural Similarity Index)	0.4383	Measures structural similarity. An SSIM of ~0.44 sim indicates that while overall structures and coarse details are retained, fine high-frequency details and exact sharp edges are slightly blurred.

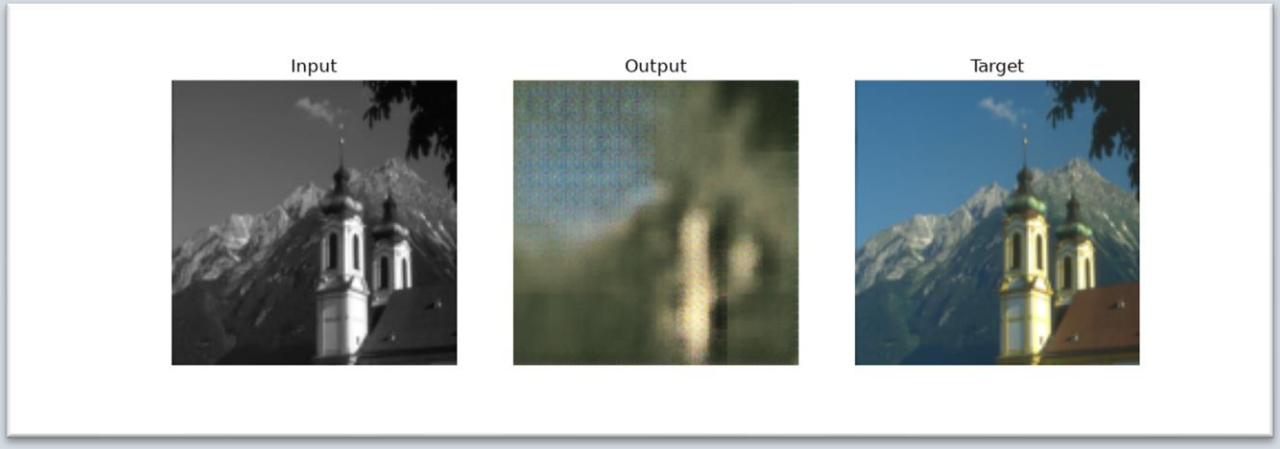
- How does the encoder-decoder structure help in image colorization?
  The encoder compresses the grayscale input into feature representations that capture shapes, edges, textures, and semantic hints. The decoder then uses these learned features to reconstruct a plausible RGB image.
  This structure is useful because the model learns a mapping from intensity patterns (grayscale) to color distributions by extracting hierarchical features and then synthesizing a full-resolution image.

- What loss function did you use and why?
  I used L1 loss between the predicted RGB image and the target RGB image.
  L1 typically produces sharper outputs than MSE for image reconstruction tasks.
  MSE tends to overly penalize large errors and often leads to blurrier/averaged colors.
  In general, L1 produced more clearly colored photos.

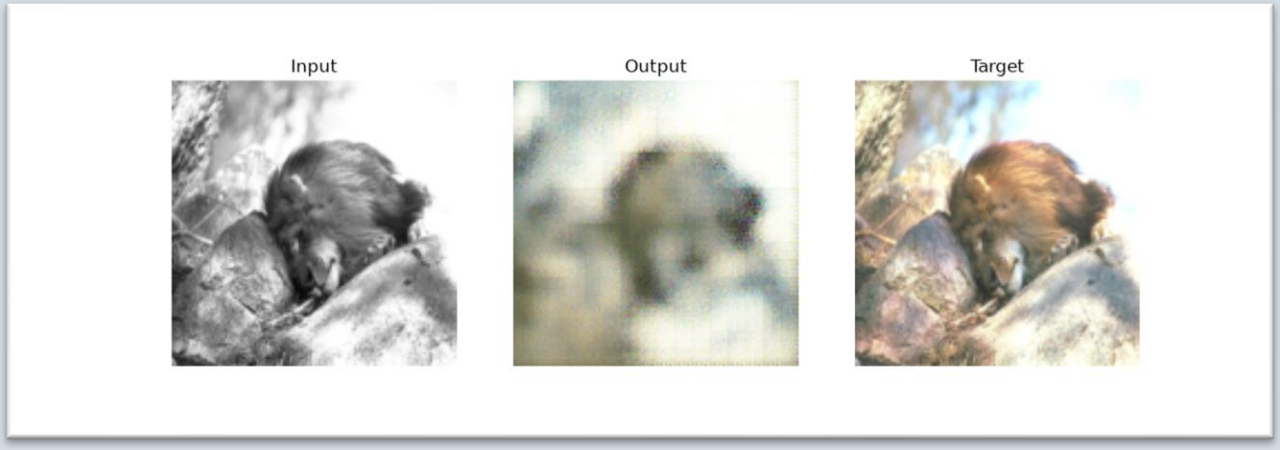
  MSE:

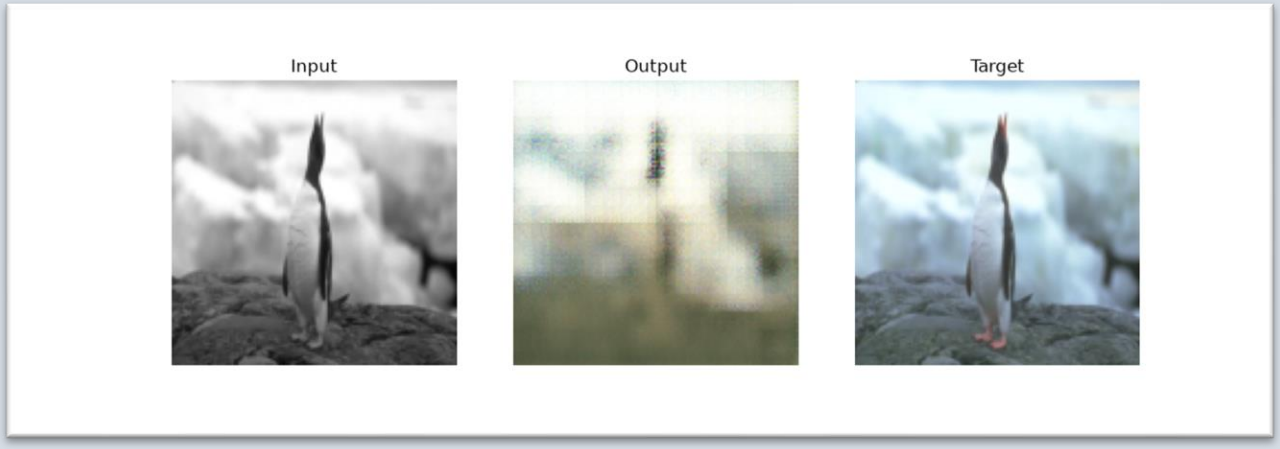


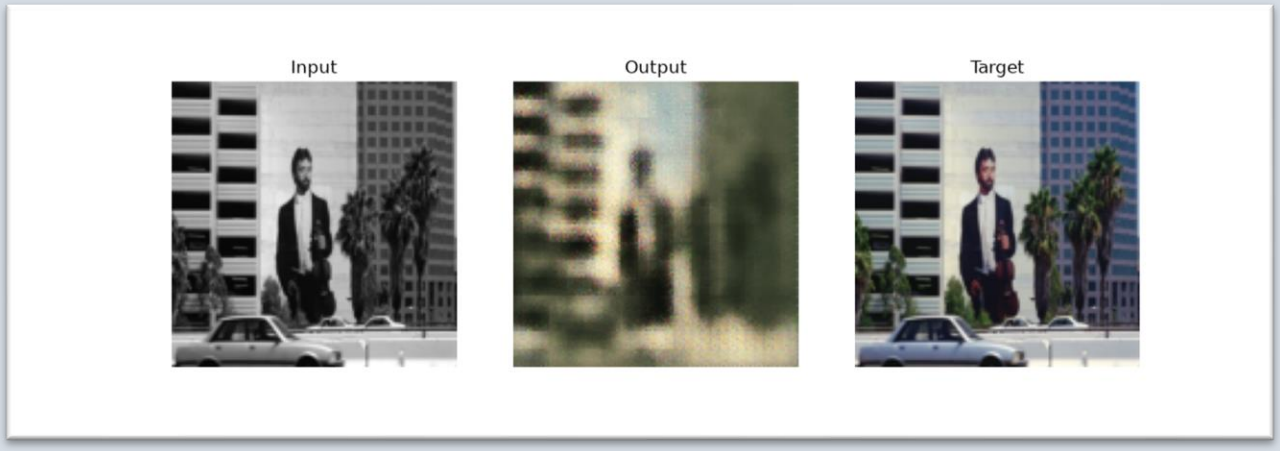
L1:

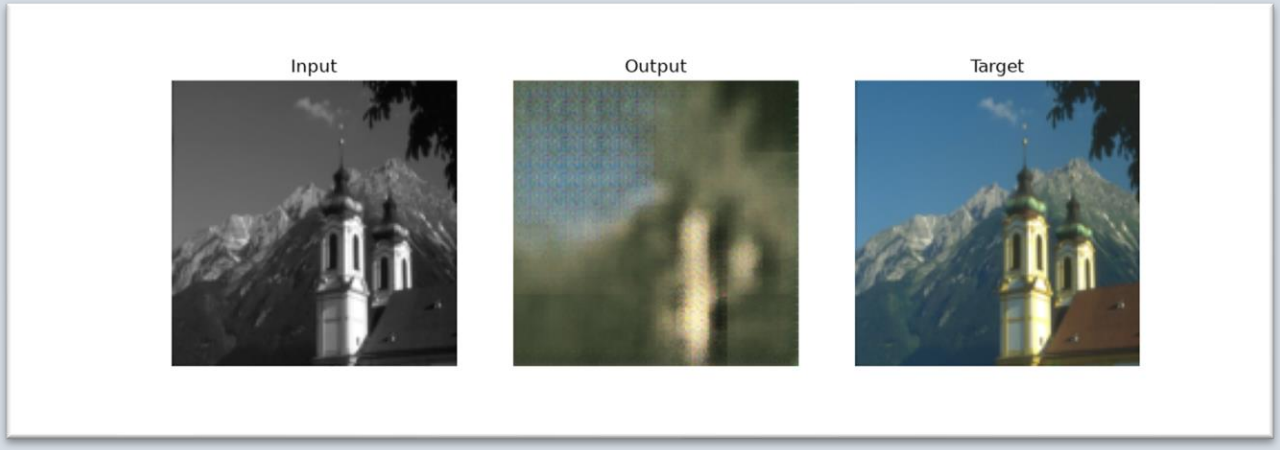


- Which test images were colorized better?









Test images featuring strong structural features and consistent, well-defined textures e.g., natural scenes like the coyote image) alongside more distinct colors yielded better results.

- Which test images were more difficult to colorize?

  AE recognized images with less distinct colors, low contrast, complex or cluttered proved more difficult. These inputs contain high-frequency information and fine edges that the autoencoder struggles to reconstruct without skip-connections. Consequently, the model produces 'averaged' color predictions and a loss of structural detail, resulting in a blurred output rather than a crisp, accurately colorized image.

- What are the main limitations of using a simple Convolutional Autoencoder for image colorization?

  Color ambiguity: Many colors are possible for the same grayscale intensity; a simple CAE often predicts "average" colors.

Loss of detail: Downsampling in the encoder can remove fine spatial details; without skip connections, details may not be recovered.

Limited semantic understanding: CAEs may not understand object identity well (e.g., knowing "this is sky → blue").

Small dataset overfitting: With limited training images, the model may not generalize well.

- How could the colorization results be improved?
  1. Using a larger dataset or adding data augmentation.
  2. Using a U-Net architecture (skip connections) to preserve details.
  3. Predicting color in Lab color space (predicting only chrominance channels) to simplify learning.
  4. Adding perceptual loss (e.g., VGG-based) to encourage visually realistic results.
  5. Using GAN-based colorization for more vivid, realistic colors.